

VERIQTA

@veriqta_



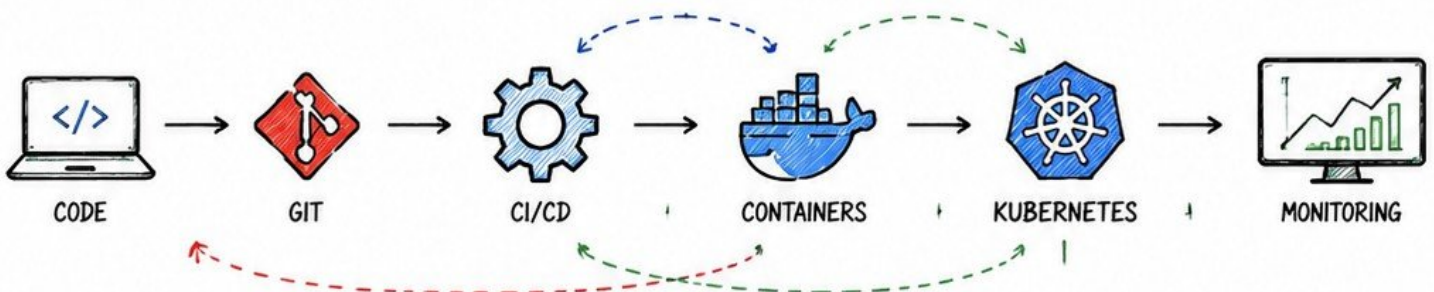
ADVANCED

DEVOPS INTERVIEW QUESTIONS



! PRODUCTION INCIDENT SCENARIOS

Real-world Scenarios. Senior-Level Thinking.
Production-Ready Mindset.



PRODUCTION
READINESS



CLOUD
SKILLS



AUTOMATION
SKILLS



SECURITY
SKILLS



ARCHITECTURE
SKILLS

PRODUCTION INCIDENT RESPONSE

Real interview questions. Real production situations.

- ★ Application outage scenarios
- ★ Troubleshooting approach
- ★ Incident management
- ★ Root cause analysis
- ★ Recovery strategy

① Q: Your production application is completely down.

Users cannot access the service. What do you do first?

- ANS:
- Check monitoring & alerts (Grafana, CloudWatch, Datadog).
 - Confirm the impact and scope (is it one service or entire system?).
 - Check status of upstream dependencies (DB, Redis, third-party APIs, DNS).
 - Verify recent changes (deployments, config changes, infra changes).
 - Start incident timeline and notify the on-call / incident channel.

INTERVIEWER LOOKS FOR:

- Calm and structured approach
- Impact assessment first
- Good use of monitoring tools
- Prioritization & communication
- Incident ownership mindset

② Q: How do you identify the root cause of a production outage?

- ANS:
- Review logs, metrics, traces across all layers.
 - Follow the request flow end-to-end (User → LB → App → DB → External).
 - Correlate events and identify the first point of failure.
 - Use tools: logs, APM, metrics, traces, dashboards.
 - Reproduce issue in staging if possible to confirm.

INTERVIEWER LOOKS FOR:

- Systematic troubleshooting
- End-to-end thinking
- Ability to drill down
- Strong debugging skills
- Data-driven decisions

③ Q: A new deployment was successful, but users are now facing errors. How do you proceed?

- ANS:
- Check application logs for exceptions and error patterns.
 - Compare metrics (error rate, latency, throughput) with previous version.
 - Validate config, environment variables, feature flags, secrets.
 - Check dependencies (DB connections, cache, queues, external services).
 - Rollback if needed to restore service quickly, then investigate.

INTERVIEWER LOOKS FOR:

- Fast but safe decision making
- Rollback strategy knowledge
- Dependency awareness
- Balancing speed vs stability
- Learn then fix approach

④ Q: How do you communicate during a production incident?

- ANS:
- Update stakeholders with clear, simple, and timely information.
 - Share impact, what we know, what we're doing, and next update time.
 - Use a single channel (Slack/Teams/Status Page).
 - Avoid technical jargon for non-technical stakeholders.
 - Document everything for post-incident review.

INTERVIEWER LOOKS FOR:

- Clear communication
- Stakeholder management
- Transparency
- Professionalism under pressure
- Documentation habit

⑤ Q: How do you recover a failed service in production?

- ANS:
- Understand the failure scope and impact.
 - Apply the quickest safe mitigation (restart, rollback, failover, scale).
 - Scale resources if needed (CPU, memory, connections).
 - Validate service health with metrics and smoke tests.
 - Monitor closely after recovery to ensure stability.

INTERVIEWER LOOKS FOR:

- Quick recovery mindset
- Risk assessment
- Monitoring after recovery
- Ensuring full restoration
- Stability focus

⑥ Q: What steps do you take after the incident is resolved?

- ANS:
- Conduct a postmortem / root cause analysis.
 - Identify what went wrong and why.
 - Document lessons learned.
 - Implement preventive actions to avoid recurrence.
 - Share postmortem with the team and stakeholders.

INTERVIEWER LOOKS FOR:

- Continuous improvement
- Ownership & accountability
- Preventive thinking
- Good documentation habits
- Team learning mindset

⑦ Q: How do you prevent similar incidents from happening again?

- ANS:
- Improve monitoring & alerting.
 - Add automated checks and health validations.
 - Implement better testing (staging, load, integration).
 - Review architecture and remove single points of failure.
 - Run regular chaos tests and DR drills.

INTERVIEWER LOOKS FOR:

- Proactive approach
- Reliability mindset
- Architecture improvement
- Learning from incidents
- Long-term thinking



KEY TAKEAWAY: Incidents are not failures, poor response is.
Respond **fast**, communicate well, learn always, and build for **resilience**.

DEPLOYMENT & CI/CD FAILURES

TOPICS COVERED:

✓ Failed deployments ✓ Rollbacks ✓ Pipeline failures ✓ Release strategies

① Q: A deployment was triggered and marked successful, but users are facing errors. How do you investigate and fix it?

ANS: • Check application logs, metrics, and error rates.
• Compare the new release with the previous stable version.
• Validate configs, environment variables, secrets, feature flags.
• Check database migrations and backward compatibility.
• If the issue is due to the new release, rollback or hotfix.
• Communicate status and monitor after fix.

INTERVIEWER LOOKS FOR:

- ★ Structured troubleshooting
- ★ Fast but safe decision making
- ★ Understanding of impact
- ★ Rollback readiness
- ★ Clear communication

② Q: How do you rollback a failed deployment in production?

ANS: • Use automated rollback (Jenkins/Kubernetes/CodePipeline) if available.
• For Kubernetes: kubectl rollout undo deployment/<name>
• For ECS/ASG: Revert to previous task definition / launch template.
• For app servers: Restore previous artifact and restart services.
• Verify rollback success with health checks and monitoring.

INTERVIEWER LOOKS FOR:

- ★ Knowledge of rollback mechanisms
- ★ Tool specific expertise
- ★ Minimizing downtime
- ★ Verification after rollback
- ★ Customer impact awareness

③ Q: What are some safe deployment strategies? When would you use each?

ANS: • Blue/Green: Two identical environments. Use for zero downtime releases.
• Canary: Release to small % of users first. Use for risk reduction.
• Rolling Update: Gradual replacement of instances/pods.
• Recreate: Stop old version before new one starts. Use for small apps.
• Dark Launch / Feature Flags: Deploy hidden, enable for users gradually.

INTERVIEWER LOOKS FOR:

- ★ Understanding of strategies
- ★ Trade-offs and use cases
- ★ Zero/low downtime mindset
- ★ Risk management
- ★ Business impact thinking

④ Q: A pipeline failed in the build stage. How do you troubleshoot?

ANS: • Check build logs for compilation/test errors.
• Verify code changes, dependencies, and build scripts.
• Reproduce the failure locally or in a container.
• Check environment, cache, and artifact repository access.
• Fix the issue and re-run the pipeline with version control.

INTERVIEWER LOOKS FOR:

- ★ Debugging skills
- ★ Reproducibility approach
- ★ Knowledge of build tools
- ★ Environment awareness
- ★ Efficient problem isolation

⑤ Q: A pipeline failed in the deploy stage. What could be the causes and how do you fix it?

ANS: • Causes: Config issues, failed health checks, insufficient permissions, bad artifacts, resource limits, network issues.
• Check deployment logs, events, and target environment health.
• Validate IAM roles, secrets, and cluster access.
• Fix the root cause and redeploy or rollback.

INTERVIEWER LOOKS FOR:

- ★ Identifying possible causes
- ★ Cloud/K8s/Infra knowledge
- ★ Security & permission awareness
- ★ Fast recovery mindset
- ★ Clear action plan

⑥ Q: How do you ensure a deployment is zero-downtime?

ANS: • Use strategies like Blue/Green, Canary, or Rolling Update.
• Ensure health checks, readiness/liveness probes are configured.
• Use load balancers to route traffic during deployment.
• Maintain backward compatibility for DB schema changes.
• Test in staging with production-like data before release.

INTERVIEWER LOOKS FOR:

- ★ Zero downtime techniques
- ★ Health check & monitoring awareness
- ★ Database change strategy
- ★ Risk mitigation
- ★ Production best practices

⑦ Q: How do you handle database migrations in production?

ANS: • Use backward compatible migrations.
• Apply migrations in steps (expand → migrate → contract).
• Test on staging with real-like data.
• Use feature flags to avoid breaking changes.
• Monitor performance during and after migration.

INTERVIEWER LOOKS FOR:

- ★ DB migration strategies
- ★ Risk mitigation
- ★ Data integrity focus
- ★ Experience with real systems
- ★ Attention to detail



KEY TAKEAWAY:

Successful deployments are not about pushing code, they are about delivering value **safely** and **reliably**.

Golden Rule:

"Automate everything you can, but always **monitor** what matters."

- * CrashLoopBackOff
- * Node failures
- * Scheduling problems
- * Networking issues

KUBERNETES PRODUCTION FAILURES

Real interview questions. Real production scenarios.

① Q: Pods are in CrashLoopBackOff status. How do you troubleshoot and fix it?

- ANS:
- Check pod description: `kubectl describe pod <pod-name>`
 - Check logs: `kubectl logs <pod-name> --previous`
 - Identify exit code, error message, and failed command.
 - Verify config, env variables, secrets, volumes, and resource limits.
 - Fix the issue and redeploy. Add readiness/liveness probes if missing.

INTERVIEWER LOOKS FOR:

- * Systematic troubleshooting
- * Understanding of pod lifecycle
- * Log analysis & root cause focus
- * Config/secret/volume awareness
- * Proactive prevention mindset

② Q: A node in the cluster is NotReady. How do you investigate?

- ANS:
- Check node status: `kubectl get nodes`
 - Describe node: `kubectl describe node <node-name>`
 - Check kubelet: `journalctl -u kubelet -f`
 - Check node resource pressure (CPU/Memory/Disk/PID).
 - Verify network connectivity & CNI status.
 - Drain the node if needed and fix the root cause.

INTERVIEWER LOOKS FOR:

- * Node lifecycle understanding
- * Kubelet & system-level debugging
- * Resource pressure awareness
- * CNI/network troubleshooting
- * Safe remediation approach

③ Q: Pods are pending and not getting scheduled. What could be the reason and how do you fix it?

- ANS:
- Check events: `kubectl get events --sort-by=.metadata.creationTimestamp`
 - Check node resources and quotas.
 - Verify node selectors, taints/tolerations, affinity/anti-affinity rules.
 - Check required PVCs, storage classes, limits, and namespaces.
 - Fix constraints or scale the cluster / adjust configs.

INTERVIEWER LOOKS FOR:

- * Scheduling flow knowledge
- * Events analysis
- * Resource & quota awareness
- * Taints/tolerations & affinity
- * Problem isolation skills

④ Q: Service is running, but pods cannot communicate with each other. How do you troubleshoot?

- ANS:
- Check if pods are in the same namespace and service discovery works.
 - Test DNS: `nslookup <service-name>` or `dig <service-name>`
 - Check NetworkPolicies blocking traffic.
 - Verify CNI plugin and pod IP connectivity.
 - Check if correct ports and selectors are configured.

INTERVIEWER LOOKS FOR:

- * Kubernetes networking model
- * DNS & service discovery
- * NetworkPolicy awareness
- * CNI understanding
- * End-to-end connectivity thinking

⑤ Q: Pod restarts intermittently with OOMKilled. How do you fix it?

- ANS:
- Check pod logs and describe for OOMKilled reason.
 - Check memory usage: `kubectl top pod <pod-name>`
 - Increase resources (requests/limits) appropriately.
 - Optimize application memory usage.
 - Add alerts for memory threshold breaches.

INTERVIEWER LOOKS FOR:

- * Resource management knowledge
- * Observability usage
- * Right-sizing mindset
- * Monitoring & alerting
- * Root cause vs symptom focus

⑥ Q: Cluster internal DNS is not resolving services.

- ANS:
- Check CoreDNS pods: `kubectl -n kube-system get pods -l k8s-app=kube-dns`
 - Check CoreDNS logs: `kubectl -n kube-system logs <coredns-pod>`
 - Verify CoreDNS configmap.
 - Check network connectivity between pods and CoreDNS.
 - Restart CoreDNS or fix upstream DNS if required.

INTERVIEWER LOOKS FOR:

- * CoreDNS knowledge
- * DNS troubleshooting flow
- * Config & logs analysis
- * Networking understanding
- * Service availability mindset

⑦ Q: Ingress is returning 502/503 errors. What do you check?

- ANS:
- Check ingress controller logs.
 - Verify backend service endpoints: `kubectl get endpoints <service-name>`
 - Check pod health and readiness probes.
 - Validate service port, targetPort, and selector.
 - Check timeouts, resource limits, and network connectivity.

INTERVIEWER LOOKS FOR:

- * Ingress flow understanding
- * Backend service troubleshooting
- * Probes & endpoint awareness
- * Layered debugging approach
- * Attention to details



KEY TAKEAWAY: Kubernetes issues are rarely random. They are signals. Follow the flow, trust the data, and fix the root cause, not the symptom.

TROUBLESHOOTING FLOW (REMEMBER):

Observe → Identify → Isolate → Fix → Validate → Prevent

- ✓ EC2 failures
- ✓ Load balancer issues
- ✓ IAM problems
- ✓ VPC troubleshooting

AWS PRODUCTION TROUBLESHOOTING

Real interview questions. Real production scenarios.

① Q: Your EC2 instance is running but your application is not accessible.

How do you troubleshoot?

- ANS:
- Check instance status & system status (AWS Health Dashboard).
 - Verify security groups allow required inbound traffic.
 - Check NACLs if traffic is blocked at subnet level.
 - Validate application is listening on correct port (netstat -tulnp).
 - Check OS firewall (ufw/iptables/firewalld).
 - Review app logs and confirm service is running.
 - Test from inside & outside the instance (telnet/curl).

INTERVIEWER LOOKS FOR:

- * Layered troubleshooting approach
- * Understanding of network flow
- * Ability to isolate the cause
- * Linux/OS level debugging
- * Clear & logical thinking

② Q: Users are getting 5xx errors from the application behind an ALB.

What could be the cause and how do you fix it?

- ANS:
- Check target group health (unhealthy targets).
 - Review target group settings: health checks, port, protocol.
 - Analyze ALB access logs / target logs.
 - Check backend application logs for errors.
 - Validate connection limits, timeouts, and deregistration delay.
 - Fix root cause (app crash, timeout) and ensure auto-healing.

INTERVIEWER LOOKS FOR:

- * Understanding of ALB architecture
- * Target group & health check knowledge
- * Log analysis skills
- * Problem isolation
- * Ownership of resolution

③ Q: You cannot SSH into an EC2 instance that was working earlier.

What do you check?

- ANS:
- Verify key pair and user (ec2-user/ubuntu/admin).
 - Check security group inbound rule for port 22.
 - Check NACL rules (inbound & outbound).
 - Ensure the instance is in the correct subnet & route table.
 - Check if the instance has a public IP / IGW / NAT.
 - Review EC2 status checks and system logs.

INTERVIEWER LOOKS FOR:

- * Systematic troubleshooting
- * Network & routing knowledge
- * Security group vs NACL understanding
- * Connectivity & OS level checks
- * Ability to think end-to-end

④ Q: An IAM user suddenly cannot access resources they were accessing yesterday.

How do you troubleshoot?

- ANS:
- Check recent policy changes (user/group/role/permissions boundary).
 - Review IAM access analyzer / policy simulator.
 - Verify MFA requirements.
 - Check SCPs if using AWS Organizations.
 - Validate temporary credentials / session expiration.
 - Check CloudTrail for denied actions & error messages.

INTERVIEWER LOOKS FOR:

- * IAM policy troubleshooting
- * Understanding of AWS Organizations
- * Security mindset
- * Use of AWS tools (CloudTrail, IAM Access Analyzer)
- * Clear root cause identification

⑤ Q: Your application in private subnets cannot access the internet.

How do you fix it?

- ANS:
- Verify route table has route to NAT Gateway (0.0.0.0/0).
 - Check if NAT Gateway is in a public subnet.
 - Validate internet gateway is attached to the VPC.
 - Ensure subnet associations are correct.
 - Check NACL rules (ephemeral ports outbound).
 - Test connectivity (curl google.com) from instance.

INTERVIEWER LOOKS FOR:

- * VPC networking fundamentals
- * Route table understanding
- * NAT Gateway knowledge
- * End-to-end connectivity testing
- * Ability to isolate network issues

⑥ Q: High latency between services in the same VPC.

How do you troubleshoot?

- ANS:
- Check security groups/NACLs for asymmetric routing.
 - Verify routing paths (traceroute/mtr).
 - Check for cross-AZ traffic charges.
 - Review network ACL & subnet configuration.
 - Check DNS resolution time.
 - Monitor network metrics (CloudWatch VPC Flow Logs, Latency).

INTERVIEWER LOOKS FOR:

- * Networking depth
- * Latency root cause analysis
- * Awareness of cost & performance
- * Use of monitoring tools
- * Thorough investigation skills

⑦ Q: You receive a "Rate exceeded" error from AWS API.

What do you do?

- ANS:
- Identify the API & service hitting the limit.
 - Check service quotas (Service Quotas console).
 - Implement exponential backoff & retries.
 - Request quota increase if needed.
 - Use caching / batching to reduce API calls.

INTERVIEWER LOOKS FOR:

- * Knowledge of AWS limits
- * Best practices for handling limits
- * Automation & resilience mindset
- * Cost & performance awareness
- * Proactive problem solving

KEY TAKEAWAY:

In production, it's never about one tool.

It's about the right approach, right order, and right mindset.

Observe → Isolate → Fix → Verify → Prevent

AWS TROUBLESHOOTING HIERARCHY (REMEMBER):



INFRASTRUCTURE AS CODE

Real interview questions. Real production scenarios.

TOPICS COVERED:

- ✓ Terraform failures
- ✓ State issues
- ✓ Drift
- ✓ Production changes

① Q: Terraform apply failed halfway through. How do you recover and bring the environment back to a consistent state?

- ANS:
- Check the error and identify the failed resource.
 - Run "terraform state list" to see what is actually created.
 - Run "terraform plan" to understand the current state vs desired state.
 - Fix the issue and re-run "terraform apply".
 - If needed, use "-target" carefully for specific resources (last resort).
 - Verify the infrastructure and test before proceeding.

INTERVIEWER LOOKS FOR:

- * Understanding of Terraform state
- * Ability to recover safely
- * Risk awareness with "-target"
- * Verification mindset
- * Focus on consistency & stability

② Q: Terraform state file is corrupted or lost. What are your options?

- ANS:
- Check if there is a backup in remote backend (S3/versioning).
 - Recover the state from backup if available.
 - If no backup, import existing resources using "terraform import".
 - Recreate state by mapping real infrastructure to Terraform.
 - Implement state locking and versioning to prevent future loss.

INTERVIEWER LOOKS FOR:

- * Knowledge of state backends
- * Disaster recovery thinking
- * Resource import skills
- * Prevention mindset
- * Understanding of state importance

③ Q: What is infrastructure drift and how do you detect and fix it?

- ANS:
- Drift happens when actual infrastructure ≠ Terraform state/config.
 - Detect drift using "terraform plan" in CI/CD or drift detection tools.
 - Review plan output to understand changes.
 - Decide: update code to match reality or enforce code to match infra.
 - Apply changes and add guardrails (policies, approvals, monitoring).

INTERVIEWER LOOKS FOR:

- * Clear understanding of drift
- * Plan-based detection
- * Decision making (reconcile vs accept)
- * Governance & guardrails
- * Continuous compliance mindset

④ Q: How do you manage Terraform state in a team environment?

- ANS:
- Store state in a remote backend (S3 + DynamoDB locking).
 - Enable state locking to avoid concurrent writes.
 - Use workspaces or multiple state files per environment.
 - Restrict access using IAM policies.
 - Follow least privilege & audit access to state.

INTERVIEWER LOOKS FOR:

- * Team collaboration best practices
- * State locking knowledge
- * Security & access control
- * Environment isolation
- * Scale & maintainability

⑤ Q: You need to make a change in a critical production environment. How do you ensure it's safe?

- ANS:
- Review the change and impact using "terraform plan".
 - Get peer/architect review for high-risk changes.
 - Use feature branches and CI validation.
 - Apply during low-traffic windows.
 - Monitor after apply and have rollback plan.

INTERVIEWER LOOKS FOR:

- * Change management process
- * Risk assessment
- * Approval workflow
- * Safe deployment mindset
- * Monitoring & rollback awareness

⑥ Q: What are common Terraform anti-patterns you avoid?

- ANS:
- Hardcoding values (use variables, tfvars, or parameter store).
 - Using "-target" regularly.
 - One huge state file for everything.
 - Ignoring provider version pinning.
 - Not using remote state or locking.

INTERVIEWER LOOKS FOR:

- * Best practices awareness
- * Experience from past issues
- * Scalable architecture thinking
- * Security & maintainability
- * Opinionated but practical mindset

⑦ Q: How do you structure Terraform code for large environments?

- ANS:
- Use modules to promote reuse and consistency.
 - Separate state per environment and/or domain.
 - Use remote state data source for dependencies.
 - Follow consistent naming & tagging standards.
 - Keep modules versioned and documented.

INTERVIEWER LOOKS FOR:

- * Modular design thinking
- * Environment isolation
- * Dependency management
- * Maintainability
- * Enterprise-grade practices



KEY TAKEAWAY: IaC is not just about writing code, it's about managing state, reducing risk, and making changes predictable and repeatable.

CHECKLIST BEFORE APPLYING TO PRODUCTION:

- | | |
|--|---|
| ☒ Plan reviewed | ☒ Impact understood |
| ☒ Peer approved | ☒ Rollback plan ready |
| ☒ Backups verified | ☒ Monitoring in place |

DOCKER PRODUCTION SCENARIOS

Real interview questions. Real production scenarios.

TOPICS COVERED:

- ✓ Container failures
- ✓ Image problems
- ✓ Runtime issues
- ✓ Security concerns

① Q: A container is running but the application inside is not responding.
How do you troubleshoot?

- ANS:
- Check container health: `docker ps`, `docker logs`, exit code.
 - Inspect container: `docker inspect <container_id>`.
 - Check resource usage: `docker stats` (CPU/Memory).
 - Verify app health endpoint or internal logs.
 - Exec into container: `docker exec -it <id> sh` to debug.
 - Check dependencies (DB, cache, API) and network connectivity.

INTERVIEWER LOOKS FOR:

- ★ Systematic troubleshooting
- ★ Knowledge of Docker commands
- ★ Ability to isolate the issue
- ★ Understanding of dependencies
- ★ Root cause mindset

② Q: A container keeps restarting with "Exit Code 137". What does it mean and how do you fix it?

- ANS:
- Exit Code 137 = OOMKilled (out of memory).
 - Check memory usage: `docker stats` or `dmesg | grep -i oom`.
 - Increase memory limit: `--memory` or update in `Compose/KBs`.
 - Optimize app memory usage / enable memory limits.
 - Add alerts for memory usage to prevent recurrence.

INTERVIEWER LOOKS FOR:

- ★ Understanding of exit codes
- ★ Linux OOM knowledge
- ★ Resource optimization
- ★ Preventive monitoring mindset
- ★ Production readiness thinking

③ Q: You pulled a new image and the container fails immediately.
What are the possible causes and how do you troubleshoot?

- ANS:
- Check image logs: `docker logs <container>`.
 - Verify entrypoint/command is correct.
 - Check environment variables and secrets.
 - Verify app dependencies & migration scripts.
 - Compare image layers: `docker history <image>`.
 - Rollback to previous working image if needed.

INTERVIEWER LOOKS FOR:

- ★ Debugging skills
- ★ Knowledge of Dockerfile concepts
- ★ Dependency awareness
- ★ Safe recovery mindset
- ★ Rollback & verification approach

④ Q: How do you reduce the size of a Docker image for production?

- ANS:
- Use multi-stage builds.
 - Use minimal base images (alpine, distroless, slim).
 - Remove unnecessary packages and build tools.
 - Use `.dockerignore` to exclude files.
 - Clean up layers (`apt-get clean`, `rm -rf /var/lib/apt/lists/*`).

INTERVIEWER LOOKS FOR:

- ★ Image optimization knowledge
- ★ Security & performance awareness
- ★ Best practice mindset
- ★ Understanding of layers
- ★ Cost & efficiency thinking

⑤ Q: A container has high CPU usage. How do you investigate and fix it?

- ANS:
- Check top processes inside container: `docker exec -it <id> top`.
 - Check CPU usage: `docker stats --no-stream`.
 - Identify any infinite loops / heavy processes / bad queries.
 - Add CPU limits: `--cpus` or `--cpu-period/--cpu-quota`.
 - Optimize code, queries, or enable caching.

INTERVIEWER LOOKS FOR:

- ★ Performance troubleshooting
- ★ Linux process knowledge
- ★ Resource management
- ★ Optimization strategies
- ★ Monitoring & alerting mindset

⑥ Q: How do you ensure container security in production?

- ANS:
- Scan images for vulnerabilities (Trivy, Clair, Gype).
 - Use non-root user in Dockerfile (`USER appuser`).
 - Minimize base image and remove unnecessary packages.
 - Use read-only filesystem where possible.
 - Manage secrets securely (Docker secrets, Vault, AWS Secrets Manager).
 - Keep images updated and patch regularly.

INTERVIEWER LOOKS FOR:

- ★ Security best practices
- ★ Vulnerability awareness
- ★ Principle of least privilege
- ★ Secrets management
- ★ Defense-in-depth mindset

⑦ Q: Containers can communicate on the same host but not across hosts.
How do you troubleshoot?

- ANS:
- Check network driver: bridge, overlay, macvlan, etc.
 - Verify port mapping: `-p host_port:container_port`.
 - Check firewall / security groups / network ACLs.
 - Validate DNS resolution between containers/hosts.
 - Test connectivity with `curl`, `ping`, `nc` from containers.

INTERVIEWER LOOKS FOR:

- ★ Networking fundamentals
- ★ Systematic troubleshooting
- ★ Understanding of Docker networks
- ★ Security group & firewall awareness
- ★ End-to-end connectivity thinking



KEY TAKEAWAY:

Containers are lightweight, but issues can be complex. Always **observe**, **isolate**, understand, and **fix** at the root. Automate, monitor, secure, and keep images lean.



DOCKER GOLDEN RULE:

Build small. Run safe.
Monitor everything.
Automate recovery.

LINUX TROUBLESHOOTING

Real interview questions. Real production scenarios.

TOPICS COVERED:

- ✓ CPU problems
- ✓ Memory leaks
- ✓ Disk issues
- ✓ Process failures

① Q: The server is extremely slow. How do you identify if the issue is CPU related and what is causing it?

- ANS:
- Check load average: `uptime` or `w` (run queue > CPU cores?).
 - Use `top`/`htop` to see high CPU processes.
 - Use `mpstat -P ALL 1` to check per-core usage.
 - Use `pidstat -u 1` to monitor CPU per process.
 - Check for runaway processes, infinite loops, or high context switching.
 - Kill/unlimit or optimize the offending process.

INTERVIEWER LOOKS FOR:

- ★ Performance troubleshooting skills
- ★ Linux command knowledge
- ★ Root cause isolation ability
- ★ Understanding of system behavior
- ★ Practical resolution approach

② Q: How do you troubleshoot high memory usage or a memory leak?

- ANS:
- Check `free -h` and `vmstat 1` to understand memory usage.
 - Use `top`/`htop` and sort by %MEM.
 - Use `ps aux --sort--%mem` to find memory hungry processes.
 - Use `smem -r` or `pmap <pid>` to analyze memory usage.
 - Check `dmesg` for OOM killer logs.
 - Restart/leak-fix the application or increase memory if required.

INTERVIEWER LOOKS FOR:

- ★ Memory analysis skills
- ★ Awareness of tools (`smem`, `pmap`)
- ★ Understanding of OOM
- ★ Root cause identification
- ★ Correct remediation

③ Q: The disk is 100% full. How do you find what is consuming space and how do you free it safely?

- ANS:
- Run `df -h` to see full partitions.
 - Use `du -sh /*` to find large directories.
 - Use `du -h --max-depth=1 /var` to dig deeper.
 - Check large logs: `/var/log`, containers, rotated logs.
 - Clean old logs, cache (`apt/yum`), temp files safely.
 - Set up log rotation and monitoring to prevent recurrence.

INTERVIEWER LOOKS FOR:

- ★ Disk usage analysis
- ★ Safe cleanup approach
- ★ Knowledge of log management
- ★ Preventive thinking
- ★ Operational awareness

④ Q: How do you troubleshoot I/O wait issues on Linux?

- ANS:
- Check `iostat -x 1` to see %iowait.
 - Use `vmstat 1` to confirm (wa column).
 - Check which process is causing I/O: `pidstat -d 1`.
 - Check disk health: `smartctl -a /dev/sdX`.
 - Look for large sequential reads/writes or high latency.
 - Optimize queries, move data, upgrade disk if needed.

INTERVIEWER LOOKS FOR:

- ★ Understanding of I/O wait
- ★ Use of monitoring tools
- ★ Storage performance knowledge
- ★ Root cause analysis
- ★ Optimization mindset

⑤ Q: A service keeps restarting or failing. How do you investigate?

- ANS:
- Check `systemctl status <service>`.
 - Check logs: `journalctl -u <service> --no-pager -n 100`.
 - Review configuration syntax and recent changes.
 - Check dependencies and ports (`ss -tulnp`).
 - Validate permissions, paths, and environment variables.
 - Fix issue and enable restart policy if appropriate.

INTERVIEWER LOOKS FOR:

- ★ Systemd & logging knowledge
- ★ Holistic investigation approach
- ★ Dependency awareness
- ★ Attention to detail
- ★ Reliability mindset

⑥ Q: How do you find and kill a zombie or defunct process?

- ANS:
- Check with `ps aux | grep Z` or `top` (look for 'Z' state).
 - Identify parent process causing it.
 - If parent is dead, zombies files re-parented to `init`, `wait` will clean them.
 - If parent is alive and not reaping, restart the parent service.
 - Use `kill -9` only if necessary.

INTERVIEWER LOOKS FOR:

- ★ Process state understanding
- ★ Linux internals awareness
- ★ Correct remediation
- ★ Safe troubleshooting
- ★ Avoiding blunt-force actions

⑦ Q: How do you troubleshoot high number of open file descriptors?

- ANS:
- Check with `lsof | wc -l` and `ulimit -n`.
 - Use `lsof | sort -k2 -n | tail` to find process using most FDs.
 - Look for apps not closing files/sockets.
 - Increase limits in `/etc/security/limits.conf` or `systemd` unit.
 - Restart service and monitor.

INTERVIEWER LOOKS FOR:

- ★ Knowledge of system limits
- ★ Use of `lsof` and `ulimit`
- ★ Application behavior awareness
- ★ Tuning & hardening mindset
- ★ Long-term monitoring

KEY TAKEAWAY:

Great engineers don't guess.
They observe, measure, isolate,
and fix the real cause.

LINUX TROUBLESHOOTING FLOW (REMEMBER):

Observe → Measure → Isolate → Analyze → Fix → Verify



NETWORKING SCENARIOS

Real interview questions. Real production scenarios.

TOPICS COVERED:

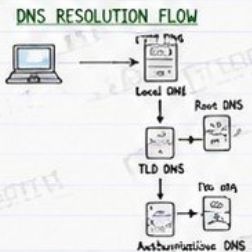
- ✓ DNS failures
- ✓ Connectivity problems
- ✓ Firewall issues
- ✓ Latency problems

① Q: Users report "site cannot be reached". Pings to IP works, but the domain name does not resolve. What do you check?

- ANS:
- Check DNS resolution: `nslookup <domain>` or `dig <domain>`.
 - Verify DNS server configuration (`/etc/resolv.conf` or system settings).
 - Check for DNS cache issues: `systemd-resolve --flush-caches` or `ipconfig /flushdns`.
 - Test with public DNS: 8.8.8.8 or 1.1.1.1.
 - Check domain DNS records (A/AAAA/CNAME) and TTL.
 - Review recent DNS changes or expiration.

INTERVIEWER LOOKS FOR:

- * DNS troubleshooting flow
- * Tools knowledge (nslookup, dig)
- * Understanding of DNS hierarchy
- * Cache & TTL awareness
- * Root cause isolation ability



② Q: A service in one server cannot connect to a database in another server on the same network. How do you troubleshoot?

- ANS:
- Ping the DB server IP.
 - Check if the port is open: `telnet <ip> <port>` or `nc -zv <ip> <port>`.
 - Verify firewall rules on both source and destination.
 - Check security groups / NACLs (cloud environments).
 - Validate the service is listening: `ss -tulpn | grep <port>`.
 - Check routing table: `ip route` or `route -n`.

INTERVIEWER LOOKS FOR:

- * Layer-by-layer approach
- * Port & service validation
- * Firewall/Security group awareness
- * Routing understanding
- * Systematic isolation mindset

CONNECTIVITY CHECKLIST

- ☐ Ping
- ☐ Port
- ☐ Firewall
- ☐ Route
- ☐ Service

③ Q: You see intermittent connection drops between client and server. What would you investigate?

- ANS:
- Check network interface errors: `ifconfig/ethtool` or `ip -s link`.
 - Look for packet loss: `ping -c 100 <ip>` or `mtr <ip>`.
 - Check network device logs: switches/routers.
 - Verify duplex mismatch, MTU issues.
 - Check for network congestion / bandwidth saturation.
 - Monitor for flapping links.

INTERVIEWER LOOKS FOR:

- * Knowledge of packet loss & latency tools
- * Network interface metrics
- * Layer 1/2/3 understanding
- * Experience with real-world issues
- * Root cause correlation

TOOLS TO USE

ping
mtr / traceroute
ip / ifstat
ethtool -S
tcpdump / Wireshark

④ Q: After a firewall rule change, applications stopped working. How do you find the issue?

- ANS:
- Review firewall rules (inbound/outbound).
 - Check rule order and default deny policies.
 - Verify allowed ports, protocols, and IP ranges.
 - Check connection tracking / state table: `conntrack -L`.
 - Test from both sides: source → destination and reverse.
 - Review firewall logs for denied traffic.

INTERVIEWER LOOKS FOR:

- * Firewall rule evaluation skills
- * Understanding of stateful inspection
- * Ability to read logs
- * Rule order & policy awareness
- * Structured testing approach

FIREWALL LOG EXAMPLE

DROP IN=eth0 OUT= SRC=10.0.1.25 DST=10.0.2.10
PROTO=TCP SPT=5321 DPT=3306
REASON=DEFAULT_DENY

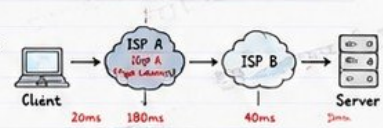
⑤ Q: High latency is reported between application servers in different regions. What would you check and how do you fix it?

- ANS:
- Check latency: `mtr --report` or ping.
 - Identify hops causing delay (high latency hop).
 - Check network path: BGP routes, ISP transit.
 - Verify bandwidth and congestion.
 - Use CDN/Global Accelerator/Anycast if applicable.
 - Optimize data transfer & reduce cross-region calls.

INTERVIEWER LOOKS FOR:

- * Latency analysis skills
- * Knowledge of routing & ISP factors
- * Optimization strategies
- * Use of performance tools
- * Cost vs performance awareness

HIGH LATENCY PATH (Example)



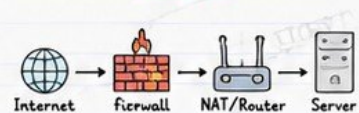
⑥ Q: A website is accessible from the office but not from the internet. What could be the problem?

- ANS:
- Check public IP / DNS is correct.
 - Verify security group / firewall allows inbound traffic.
 - Check NAT / Port Forwarding rules.
 - Validate the service is bound to public interface (0.0.0.0).
 - Check ISP / upstream blocks.

INTERVIEWER LOOKS FOR:

- * Exposure & access control knowledge
- * NAT & Port forwarding understanding
- * Service binding awareness
- * Security group & firewall logic
- * End-to-end visibility

ACCESS PATH



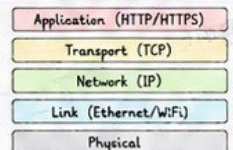
⑦ Q: DNS resolves but the website is still not loading. What's your next step?

- ANS:
- Check HTTP(S) connectivity: `curl -I https://domain.com`
 - Verify SSL/TLS handshake: `openssl s_client -connect domain.com:443`
 - Check web server logs for errors.
 - Validate load balancer health & backend availability.
 - Use browser dev tools / network tab.

INTERVIEWER LOOKS FOR:

- * Beyond DNS thinking
- * Application layer troubleshooting
- * SSL/TLS knowledge
- * Load balancer awareness
- * Layered investigation

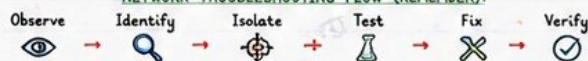
LAYERED MODEL



KEY TAKEAWAY:

Networking issues are invisible until you observe, measure, isolate, and verify. Always validate each layer.

NETWORK TROUBLESHOOTING FLOW (REMEMBER):



OBSERVABILITY QUESTIONS

Real interview questions. Real production scenarios.

TOPICS COVERED:

- ✓ Metrics
- ✓ Logs
- ✓ Tracing
- ✓ Alerting

① Q: Your application latency suddenly increased. How do you use metrics to find the root cause?

- ANS:
- Start with RED/USE method: Rate, Errors, Duration / Utilization, Saturation, Errors.
 - Check latency (p50, p95, p99) and compare with baseline.
 - Drill down by service, endpoint, region, version.
 - Correlate with dependencies: DB, cache, queue, external APIs.
 - Check resource metrics: CPU, memory, GC, disk I/O, network.
 - Use dashboards & SLI/SLO to validate impact.

INTERVIEWER LOOKS FOR:

- Understanding of RED/USE & SLIs
- Ability to go from symptoms to cause
- Correlation across components
- Use of percentiles & baselines
- Observability-driven thinking

RED METHOD

R	Rate	
E	Errors	
D	Duration	

② Q: You have high error rates but logs show nothing useful. What will you do?

- ANS:
- Improve logging: add structured logs with correlation/request IDs.
 - Increase log level temporarily (with caution).
 - Check centralized logs (ELK/Loki/CloudWatch/Datadog).
 - Search by trace ID, user ID, or timestamp.
 - Add contextual fields: service, env, version, endpoint, latency.
 - Reproduce in lower env and compare logs.

INTERVIEWER LOOKS FOR:

- Logging best practices knowledge
- Structured logging awareness
- Correlation & traceability
- Systematic troubleshooting
- Balance between signal & noise

GOOD LOG =

- ✓ Structured
- ✓ Contextual
- ✓ Correlated
- ✓ Actionable



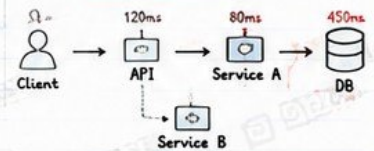
③ Q: How do you use distributed tracing to troubleshoot a slow request?

- ANS:
- Get the trace ID from logs / error response / APM.
 - Open trace in tool (Jaeger, Zipkin, Datadog, AWS X-Ray).
 - Follow the request flow across all services.
 - Identify the span with highest latency.
 - Check span details: DB calls, external APIs, queues.
 - Validate with logs & metrics to confirm the bottleneck.

INTERVIEWER LOOKS FOR:

- Understanding of distributed tracing
- Ability to read traces
- Identifying latency hotspots
- Correlation with logs/metrics
- End-to-end thinking

TRACE FLOW (Example)



④ Q: An alert fired at 3AM. What do you check first?

- ANS:
- Check alert details: metric, threshold, duration.
 - Check dashboard linked to the alert.
 - Validate if it's a real issue or a false positive.
 - Check recent deployments / changes.
 - Check related alerts (correlation).
 - Acknowledge, communicate, and start investigation.

INTERVIEWER LOOKS FOR:

- Alert triage process
- Avoiding alert fatigue
- Change correlation
- Communication mindset
- Structured response

ALERT TRIAGE CHECKLIST

- 📄 Summary
- 📊 Dashboard
- 🔗 Recent Changes
- 🔔 Related Alerts
- ❗ Impact

⑤ Q: How do you design meaningful alerts?

- ANS:
- Alert on user impact (SLIs), not just system internals.
 - Use proper thresholds and baseline.
 - Configure for severity, priority, and routing
 - Avoid noisy alerts: use rate of change, anomaly detection.
 - Add runbook links and clear descriptions.
 - Review & tune alerts regularly.

INTERVIEWER LOOKS FOR:

- Alerting best practices
- SLI/SLO awareness
- Reducing noise
- Clear ownership & runbooks
- Continuous improvement

GOOD ALERT IS:

- ✓ Actionable
- ✓ Relevant
- ✓ Timely
- ✓ Owned
- ✓ Documented



⑥ Q: Logs are growing too fast and storage is expensive. How do you handle this?

- ANS:
- Implement log retention & lifecycle policies.
 - Use log levels: DEBUG in lower env only.
 - Filter & drop unnecessary logs at source.
 - Aggregate logs and keep summaries.
 - Use cheaper storage tiers (S3 Glacier, GCS Coldline).
 - Monitor log volume & set budgets/alerts.

INTERVIEWER LOOKS FOR:

- Cost awareness
- Logging strategy
- Retention & governance
- Trade-offs understanding
- Monitoring observability data

LOG MANAGEMENT FLOW



⑦ Q: How do you know if your system is healthy overall?

- ANS:
- Define SLIs (latency, availability, error rate, saturation).
 - Track SLOs and error budgets.
 - Use dashboards for RED/USE + business metrics.
 - Correlate metrics, logs, traces, and alerts.
 - Regularly review capacity and trends.
 - Run chaos tests and validate recoverability.

INTERVIEWER LOOKS FOR:

- SRE mindset
- SLI/SLO & error budgets
- Holistic observability
- Proactive approach
- Reliability focus

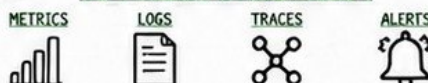
HEALTH CHECK =



KEY TAKEAWAY:

Observability is knowing what's happening, why it's happening, and what to do about it - before your users tell you.

THE 4 PILLARS OF OBSERVABILITY



SECURITY & DEVSECOPS

Real interview questions. Real production scenarios.

TOPICS COVERED:

- ✓ Secrets exposure
- ✓ Vulnerabilities
- ✓ Access control
- ✓ Security incidents

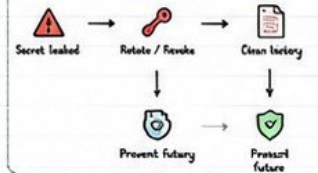
① Q: A secret was exposed in a Git repository. What do you do immediately?

- ANS:
- Revoke/rotate the exposed secret immediately.
 - Remove the secret from Git history (BFG or git filter-repo).
 - Invalidate any tokens/keys/credentials that may be compromised.
 - Scan for other occurrences and ensure no other leaks.
 - Communicate with stakeholders and document the incident.
 - Implement secret scanning & pre-commit hooks to prevent reoccurrence.

INTERVIEWER LOOKS FOR:

- Incident response mindset
- Understanding of blast radius
- Ability to take immediate action
- Preventive controls thinking
- Communication & documentation

SECRETS EXPOSURE FLOW



② Q: How do you find and fix vulnerabilities in a container image?

- ANS:
- Scan image using tools: Trivy, Gypse, Clair, or Anchore.
 - Review CVE severity and exploitability.
 - Update base image to latest secure version.
 - Patch vulnerable packages or remove unnecessary packages.
 - Rebuild and rescan to verify.
 - Automate scans in CI/CD and set failure gates.

INTERVIEWER LOOKS FOR:

- Knowledge of vulnerability scanning
- Risk prioritization
- Remediation approach
- Automation mindset
- Secure image lifecycle

VULNERABILITY REMEDIATION



③ Q: A user reports they cannot access a resource they should have access to. How do you troubleshoot?

- ANS:
- Verify user identity and group membership.
 - Review IAM roles, policies, and permissions.
 - Check resource-based policies.
 - Inspect service control policies (SCPs) or organization restrictions.
 - Review logs for access denied events.
 - Apply least privilege and fix misconfigurations.

INTERVIEWER LOOKS FOR:

- IAM & access control knowledge
- Systematic troubleshooting
- Understanding of policy evaluation
- Cloud security fundamentals
- Least privilege mindset

AWS IAM ACCESS FLOW



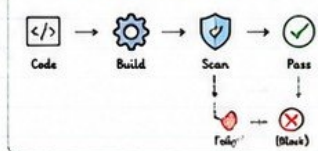
④ Q: Your pipeline failed a security scan. How do you handle it?

- ANS:
- Review scan report and understand the findings.
 - Prioritize issues based on severity and exploitability.
 - Fix the issue in code, dependencies, or configuration.
 - Re-run scans to validate.
 - Document the fix and update security controls if needed.
 - Educate the team and prevent similar issues.

INTERVIEWER LOOKS FOR:

- Security-first mindset
- Risk-based prioritization
- Continuous improvement
- Automation & quality gates
- Team collaboration

SECURE PIPELINE EXAMPLE



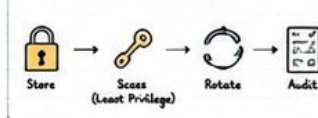
⑤ Q: How do you manage secrets securely in production?

- ANS:
- Use a secrets manager (AWS Secrets Manager, HashiCorp Vault, Azure Key Vault).
 - Never store secrets in code, images, or environment variables.
 - Use IAM roles/roles for service accounts (IRSA) where possible.
 - Rotate secrets regularly.
 - Restrict access with least privilege and audit usage.

INTERVIEWER LOOKS FOR:

- Knowledge of secret management tools
- Best practices understanding
- Least privilege & rotation
- Compliance & audit awareness
- Production readiness

SECRET MANAGEMENT



⑥ Q: You detected unusual activity in your production environment. What is your response?

- ANS:
- Detect: Alert triggered / anomaly noticed.
 - Assess: Validate the alert and scope the impact.
 - Contain: Isolate affected systems / revoke credentials.
 - Eradicate: Remove threat, patch vulnerabilities.
 - Recover: Restore normal operations and monitor.
 - Post-mortem: Document, learn, and improve.

INTERVIEWER LOOKS FOR:

- Incident response process
- NIST-style thinking (Detect-Respond-Recover)
- Calm under pressure
- Communication & coordination
- Continuous learning

INCIDENT RESPONSE LIFECYCLE



⑦ Q: How do you enforce security as code?

- ANS:
- Use IaC tools with security scanning (Checkov, tfsec, cfn-nag).
 - Implement policy as code (OPA/Conftest, Sentinel).
 - Fail builds on policy violations.
 - Maintain version-controlled policies and exceptions.
 - Continuously review and update rules.

INTERVIEWER LOOKS FOR:

- DevSecOps mindset
- Policy as code knowledge
- Automation & governance
- Shift-left security
- Consistency & compliance

SECURITY AS CODE PIPELINE



KEY TAKEAWAY:

Security is everyone's responsibility.
Build secure, monitor continuously,
and respond quickly.

DEVSECOPS PRINCIPLES

Security is not a product, it's a process. It's a continuous job. — DevSecOps Mindset

CLOUD ARCHITECTURE QUESTIONS

Real interview questions. Real production scenarios.

TOPICS COVERED:

- ✓ High availability
- ✓ Scalability
- ✓ Disaster recovery
- ✓ Multi-region design

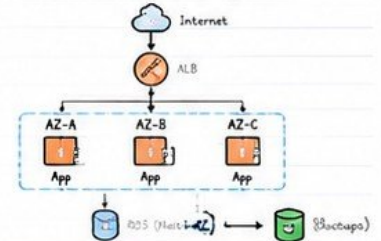
① Q: How do you design a highly available application in the cloud?

- ANS:
- Distribute across multiple AZs.
 - Use load balancers to distribute traffic.
 - Remove single points of failure.
 - Use managed services with built-in HA (RDS Multi-AZ, ELB, etc).
 - Implement health checks and auto-recovery.
 - Test failure scenarios regularly.

INTERVIEWER LOOKS FOR:

- Understanding of AZs & fault tolerance
- Load balancing knowledge
- Elimination of SPOF
- Use of managed services
- Resilience mindset

HIGH AVAILABILITY ARCHITECTURE



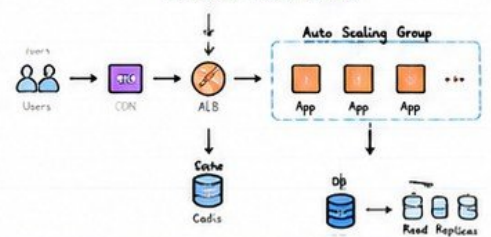
② Q: How do you design a system that can scale to millions of users?

- ANS:
- Use horizontal scaling (add more instances/containers).
 - Stateless application architecture.
 - Auto Scaling Groups / Kubernetes HPA.
 - Use caching (Redis, ElastiCache, CDN).
 - Database scaling: read replicas, sharding if needed.
 - Perform load testing and capacity planning.

INTERVIEWER LOOKS FOR:

- Horizontal vs vertical scaling
- Stateless design
- Auto scaling concepts
- Caching strategies
- Database scaling knowledge
- Capacity planning awareness

SCALABLE ARCHITECTURE



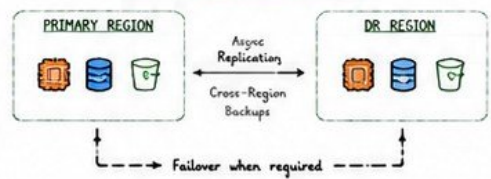
③ Q: How do you handle disasters and ensure business continuity?

- ANS:
- Define RPO (Recovery Point Objective) and RTO (Recovery Time Objective).
 - Automated backups and regular restores.
 - Multi-AZ for high availability.
 - Cross-region backups / replication.
 - Infrastructure as Code for quick rebuild.
 - Documented runbooks & regular DR drills.

INTERVIEWER LOOKS FOR:

- RPO/RTO understanding
- Backup & restore strategy
- DR planning
- Automation & IaC usage
- Testing & documentation

DISASTER RECOVERY STRATEGY



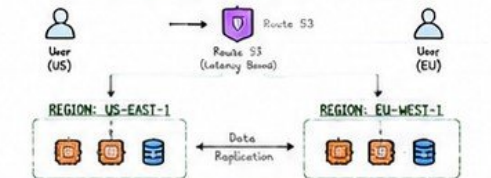
④ Q: How do you design a multi-region architecture?

- ANS:
- Deploy to multiple regions close to users.
 - Use DNS-based routing (Route 53 latency / geolocation).
 - Active-Active or Active-Passive strategy.
 - Replicate data across regions.
 - Handle eventual consistency and conflicts.
 - Monitor globally and test failover.

INTERVIEWER LOOKS FOR:

- DNS routing strategies
- Active-active vs active-passive
- Data replication patterns
- Consistency trade-offs
- Global monitoring & testing

MULTI-REGION ARCHITECTURE (Active-Active)



⑤ Q: What is the difference between High Availability and Disaster Recovery?

- ANS:
- High Availability (HA): Keep the system running during component failures. Local to a region (e.g., Multi-AZ).
 - Disaster Recovery (DR): Recover the system after a major outage. Involves multiple regions.

KEY DIFFERENCE

HIGH AVAILABILITY	DISASTER RECOVERY
Minimize downtime	Recover after disaster
Within a region	Across regions
Example: Multi-AZ	Example: Cross-region backup
Goal: Stay up	Goal: Recover

WHEN TO USE WHAT?

- Use HA for component failures
- Use DR for regional disasters
- Use Both for full resilience

⑥ Q: How do you reduce costs while keeping the architecture reliable?

- ANS:
- Right-size resources.
 - Use managed services.
 - Auto Scaling (scale down when not needed).
 - Use Spot/Reserved Instances where applicable.
 - Implement lifecycle policies (S3, EBS snapshots).
 - Monitor & optimize continuously.

INTERVIEWER LOOKS FOR:

- Cost optimization mindset
- Use of cloud native services
- Autoscaling & efficiency
- Understanding of pricing models
- Continuous improvement

COST OPTIMIZATION TIPS



⑦ Q: What questions would you ask before designing a cloud solution?

- ANS:
- What are the business goals and constraints?
 - What is the expected load and growth?
 - What are the availability and performance requirements?
 - What are the compliance and security requirements?
 - What is the budget and timeline?
 - What are the recovery and backup requirements?

INTERVIEWER LOOKS FOR:

- Requirement gathering skills
- Business understanding
- Non-functional requirements
- Risk & compliance awareness
- Holistic thinking

DESIGN THINKING CHECKLIST

- Business Goals ✓
- Workload & Growth ✓
- Availability & Performance ✓
- Security & Compliance ✓
- Budget & Timeline ✓
- DR & Backup Needs ✓

KEY TAKEAWAY:

Great cloud architectures are built on resilience, scalability, and simplicity. Plan for failure. Design for scale.

CLOUD ARCHITECT'S MINDSET

Reliable. Scalable. Secure. Cost-effective.
Automated. Observable. Recoverable.

DATABASE RELIABILITY

Real interview questions. Real production scenarios.

TOPICS COVERED:

- ✓ Database outages
- ✓ Performance issues
- ✓ Backup recovery
- ✓ Scaling

① Q: Your production database is down and users cannot connect.

What do you do first?

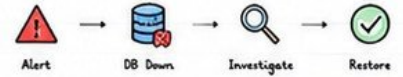
ANS:

- Check monitoring alerts and gather quick facts.
- Verify DB instance status (is it down or restricting?).
- Check recent changes/deployments or maintenance.
- Look at error logs and system metrics.
- Communicate status, set expectation & start incident bridge.

INTERVIEWER LOOKS FOR:

- Incident response mindset
- Prioritization
- Root cause investigation approach
- Communication
- Calm under pressure

DATABASE OUTAGE SCENARIO



② Q: A query is running very slow and impacting the application.

How do you troubleshoot?

ANS:

- Identify the slow query (slow query log / APM).
- Analyze execution plan (EXPLAIN / ANALYZE).
- Check indexes, joins, and table scans.
- Validate data volume and query statistics.
- Optimize query, add indexes or rewrite if necessary.

INTERVIEWER LOOKS FOR:

- Query analysis skills
- Indexing knowledge
- Performance tuning mindset
- Use of DB tools
- Systematic approach

PERFORMANCE TROUBLESHOOTING FLOW



③ Q: How do you ensure your database can recover from failures?

ANS:

- Implement regular automated backups (full + point-in-time).
- Test restore process regularly.
- Use replication or snapshots for quick recovery.
- Document Recovery Point Objective (RPO) and Recovery Time Objective (RTO).
- Keep backups in a separate region/account.

INTERVIEWER LOOKS FOR:

- Backup strategy knowledge
- RPO/RTO understanding
- Testing discipline
- Disaster recovery awareness
- Data safety mindset

BACKUP & RECOVERY STRATEGY



④ Q: Your database storage is growing fast (90% full). What do you do?

ANS:

- Identify top space consumers (tables, indexes, logs).
- Archive or purge old data.
- Optimize tables and indexes.
- Increase storage if needed and plan capacity.
- Set up alerts to prevent future issues.

INTERVIEWER LOOKS FOR:

- Capacity planning
- Data lifecycle management
- Proactive monitoring
- Problem prevention
- Practical solutions

STORAGE GROWTH CHECKLIST

- ✓ Find top space consumers
- ✓ Clean up / Archive data
- ✓ Optimize indexes
- ✓ Increase storage
- ✓ Set alerts

⑤ Q: How do you scale a database for more traffic?

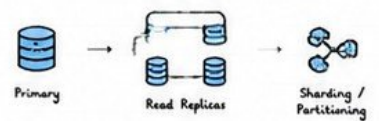
ANS:

- Add read replicas for read-heavy workloads.
- Use connection pooling to manage connections.
- Partition / Shard large tables.
- Optimize queries and indexes.
- Consider managed services that support auto-scaling.

INTERVIEWER LOOKS FOR:

- Scaling strategies
- Read vs Write workload understanding
- Architecture awareness
- Trade-off evaluation
- Cost & performance balance

SCALING OPTIONS



⑥ Q: How do you detect and prevent database performance issues?

ANS:

- Monitor key metrics: CPU, Memory, Disk I/O, Connections, Query time.
- Set alerts and thresholds.
- Review slow query logs and performance dashboards.
- Perform regular maintenance (VACUUM, ANALYZE, REINDEX).
- Capacity plan based on trends.

INTERVIEWER LOOKS FOR:

- Monitoring & observability
- Proactive approach
- Performance tuning knowledge
- Maintenance practices
- Data-driven decisions

KEY METRICS TO MONITOR



⑦ Q: A recent deployment caused database errors.

How do you handle it?

ANS:

- Check error logs and timeline of deployment.
- Reproduce issue in staging if possible.
- Rollback if necessary and safe.
- Fix root cause and test thoroughly.
- Communicate updates and document the incident.

INTERVIEWER LOOKS FOR:

- Incident handling
- Rollback mindset
- Root cause focus
- Communication
- Continuous improvement

INCIDENT RESPONSE FLOW



KEY TAKEAWAY:

Reliable databases are built on visibility, preparation, and continuous improvement. Plan ahead, monitor always, test often!

DB RELIABILITY MINDSET



PERFORMANCE ENGINEERING

Real interview questions. Real production scenarios.

TOPICS COVERED:

- ✓ Bottleneck analysis
- ✓ Capacity planning
- ✓ Optimization

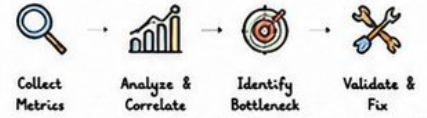
① Q: How do you identify the bottleneck in a system?

- ANS:
- Collect metrics across all layers (CPU, Memory, I/O, Network, DB).
 - Use APM, profiling, and tracing to find slow components.
 - Compare resource utilization vs saturation.
 - Look for high wait times, queues, or error rates.
 - Isolate the component causing the highest latency or lowest throughput.

INTERVIEWER LOOKS FOR:

- Systematic approach
- Knowledge of metrics & tools
- Root cause isolation
- Layer-by-layer thinking
- Data-driven mindset

BOTTLENECK ANALYSIS FLOW



② Q: What tools do you use for performance analysis?

- ANS:
- APM: New Relic, Datadog, Dynatrace, AppDynamics
 - Profiling: pprof, YourKit, JProfiler
 - Monitoring: Prometheus + Grafana
 - Tracing: Jaeger, Zipkin, AWS X-Ray
 - Load Testing: k6, JMeter, Locust, Gatling
 - Logs & Metrics: ELK Stack, CloudWatch

INTERVIEWER LOOKS FOR:

- Tool awareness
- When and why to use each
- Breadth across layers
- Cloud-native knowledge
- Practical experience

TOOLS BY LAYER



③ Q: How do you analyze a slow API endpoint?

- ANS:
- Check response time percentiles (p50, p95, p99).
 - Trace the request path using distributed tracing.
 - Identify slow external calls, DB queries, or downstream services.
 - Check database query plans and indexes.
 - Optimize code, queries, or configuration based on findings.

INTERVIEWER LOOKS FOR:

- End-to-end thinking
- Tracing & correlation
- DB query analysis
- Prioritization of fixes
- Continuous improvement

SLOW API ANALYSIS FLOW



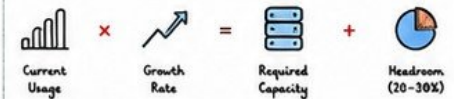
④ Q: How do you perform capacity planning?

- ANS:
- Understand current usage and growth trends.
 - Forecast future demand (users, traffic, data).
 - Determine resource requirements (CPU, RAM, Storage, Network).
 - Consider peaks, seasonality, and business events.
 - Plan headroom (20-30%) and review regularly.

INTERVIEWER LOOKS FOR:

- Forecasting skills
- Understanding of resources
- Business alignment
- Headroom & risk management
- Review and adaptation

CAPACITY PLANNING FRAMEWORK



⑤ Q: How do you test performance before going to production?

- ANS:
- Define performance goals (SLA, response time, throughput).
 - Create realistic test scenarios and test data.
 - Run load, stress, spike, and endurance tests.
 - Identify bottlenecks and optimize.
 - Re-test and validate before release.

INTERVIEWER LOOKS FOR:

- Test strategy
- Types of performance tests
- Realistic scenarios
- Iterative improvement
- Confidence in release

PERFORMANCE TESTING PYRAMID



⑥ Q: How do you common performance bottlenecks?

- ANS:
- CPU: High utilization, inefficient code, infinite loops.
 - Memory: Leaks, high garbage collection, swapping.
 - Disk I/O: Slow reads/writes, high latency storage.
 - Network: High latency, packet loss, bandwidth saturation.
 - Database: Slow queries, missing indexes, lock contention.

INTERVIEWER LOOKS FOR:

- Awareness of bottleneck types
- Layer-by-layer understanding
- Root cause thinking
- Monitoring correlation
- Practical experience

COMMON BOTTLENECKS



⑦ Q: How do you optimize a system for better performance?

- ANS:
- Optimize code and algorithms.
 - Add caching (Redis, CDN, in-memory).
 - Optimize database (indexes, query tuning).
 - Use async processing and queues.
 - Scale horizontally and use load balancers.

INTERVIEWER LOOKS FOR:

- Optimization techniques
- Trade-off understanding
- Caching & async patterns
- Scaling strategies
- Continuous monitoring

OPTIMIZATION TOOLKIT



KEY TAKEAWAY:

Performance is not a one-time task.
It's a continuous cycle of: Measure → Analyze → Optimize → Validate → Repeat.

PERFORMANCE ENGINEERING MINDSET



MONITORING & INCIDENT MANAGEMENT

Real interview questions. Real production scenarios.

TOPICS COVERED:

- ✓ Alerts
- ✓ On-call scenarios
- ✓ Postmortems

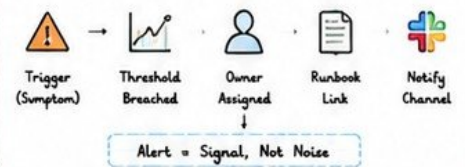
① Q: How do you design effective alerts?

- ANS:
- Alert on symptoms (user impact), not on infrastructure.
 - Set meaningful thresholds and avoid noise.
 - Use multiple conditions / multi-window alerts.
 - Add context: runbook link, dashboard, owner.
 - Review and tune alerts regularly.

INTERVIEWER LOOKS FOR:

- Alerting best practices
- Noise reduction mindset
- Understanding of impact
- Clarity and ownership
- Continuous improvement

GOOD ALERT DESIGN EXAMPLE



② Q: You get paged at 2 AM. What's your first move?

- ANS:
- Acknowledge the alert.
 - Assess severity and customer impact.
 - Check dashboards and recent changes.
 - Engage team if needed.
 - Communicate status and start investigation.

INTERVIEWER LOOKS FOR:

- Calm and structured approach
- Prioritization
- Communication
- Impact assessment
- Team collaboration

ON-CALL FIRST 5 MINUTES



③ Q: How do you handle a high-severity incident?

- ANS:
- Declare incident and communicate early.
 - Assign roles (commander, scribe, ops, comms).
 - Focus on mitigation first, then root cause.
 - Update stakeholders regularly.
 - Resolve, document, and learn.

INTERVIEWER LOOKS FOR:

- Incident management process
- Leadership and ownership
- Prioritization of mitigation
- Stakeholder communication
- Learning mindset

INCIDENT RESPONSE FLOW



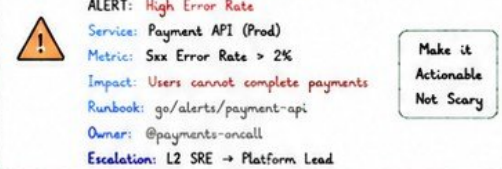
④ Q: What should be included in a good alert?

- ANS:
- What happened (metric & threshold).
 - Where (service, environment, region).
 - Impact (who is affected).
 - What to do (runbook link, commands).
 - Who to contact (on-call, escalation path).

INTERVIEWER LOOKS FOR:

- Alert context
- Actionable information
- Ownership clarity
- Runbook usage
- User impact awareness

ALERT ANATOMY



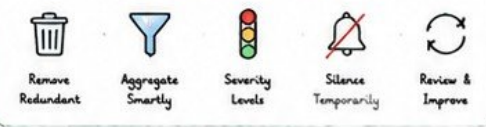
⑤ Q: How do you reduce alert fatigue?

- ANS:
- Remove redundant and low-value alerts.
 - Aggregate alerts by service/impact.
 - Use proper severity levels.
 - Silence known issues with expiry.
 - Continuously review and improve.

INTERVIEWER LOOKS FOR:

- Noise reduction strategies
- Tooling & configuration
- Risk awareness
- Proactive mindset
- Continuous optimization

ALERT FATIGUE KILLERS



⑥ Q: What is an on-call rotation best practice?

- ANS:
- Fair rotation and follow-the-sun when global.
 - Clear escalation and handoff process.
 - Adequate tooling and documentation.
 - Limit pages during non-working hours.
 - Provide support, training, and rest.

INTERVIEWER LOOKS FOR:

- Operational maturity
- Team well-being awareness
- Process & tooling
- Escalation clarity
- Sustainable operations

ON-CALL BEST PRACTICES



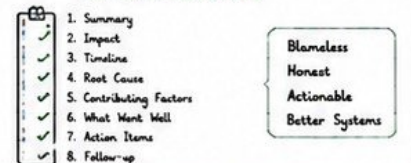
⑦ Q: How do you run a blameless postmortem?

- ANS:
- Focus on system failures, not personal mistakes.
 - Follow a structured format.
 - Identify root causes and contributing factors.
 - Define actionable improvement items.
 - Track actions to closure and share learnings.

INTERVIEWER LOOKS FOR:

- Blameless culture
- Root cause analysis
- Actionable outcomes
- Follow-through
- Continuous learning

POSTMORTEM STRUCTURE



KEY TAKEAWAY:

Monitoring tells you when.
Incident management helps you act.
Postmortems ensure you get better.
Observe. Respond. Improve.

INCIDENT LIFECYCLE



PLATFORM ENGINEERING

Real interview questions. Real production scenarios.

TOPICS COVERED:

- ✓ Internal developer platforms
- ✓ Self-service infrastructure
- ✓ Developer experience

① Q: What is an Internal Developer Platform (IDP)?

- ANS:
- A curated set of tools, services, and workflows.
 - Abstracts infrastructure complexity.
 - Enables developers to ship software faster.
 - Built on self-service, automation, and standardization.
 - Improves security, compliance, and developer experience.

INTERVIEWER LOOKS FOR:

- Understanding of IDP purpose
- Developer productivity mindset
- Abstraction & standardization
- Reusability & composability
- Measurable outcomes

WHAT IS AN IDP?



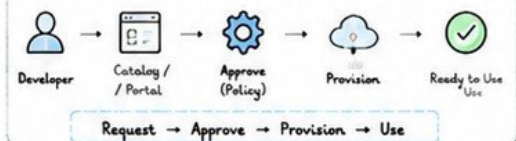
② Q: How does a self-service infrastructure improve productivity?

- ANS:
- Developers provision what they need, when they need it.
 - Reduces ticket dependency and context switching.
 - Uses golden paths, templates, and guardrails.
 - Provides safe defaults with policy enforcement.
 - Shortens delivery time from days to minutes.

INTERVIEWER LOOKS FOR:

- Automation mindset
- Guardrails over governance
- Balance of freedom & control
- Speed with compliance
- Real-world outcomes

SELF-SERVICE INFRASTRUCTURE



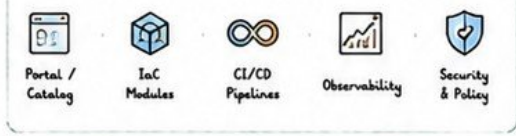
③ Q: What are key building blocks of a developer platform?

- ANS:
- Self-service portal / catalog.
 - Infrastructure as Code (IaC) modules.
 - CI/CD templates & pipelines.
 - Observability & logging.
 - Security, compliance & policy as code.

INTERVIEWER LOOKS FOR:

- End-to-end platform view
- Key components knowledge
- Integration of tools
- Security & compliance
- Developer-centric design

CORE BUILDING BLOCKS



④ Q: How do you ensure good developer experience (DX)?

- ANS:
- Make it easy to discover, adopt, and use.
 - Provide clear docs, examples, and templates.
 - Fast feedback loops and reliable tools.
 - Consistent UX and naming conventions.
 - Collect feedback and iterate continuously.

INTERVIEWER LOOKS FOR:

- Empathy for developers
- UX & documentation importance
- Feedback & continuous improvement
- Metrics driven DX
- Iterative mindset

DEVELOPER EXPERIENCE PRINCIPLES



⑤ Q: How do you secure a developer platform?

- ANS:
- Apply least privilege access.
 - Use policy-as-code (OPA, Sentinel).
 - Scan IaC, images, and dependencies.
 - Secrets management & encryption.
 - Continuous compliance and audit trails.

INTERVIEWER LOOKS FOR:

- Security by design
- Policy enforcement
- Tooling & automation
- Compliance awareness
- Risk reduction mindset

SECURITY LAYERS



⑥ Q: How do you measure success of a platform?

- ANS:
- Adoption rate and active users.
 - Lead time to provision.
 - Change failure rate and MTTR.
 - Developer satisfaction (NPS / surveys).
 - Cost savings and resource efficiency.

INTERVIEWER LOOKS FOR:

- Relevant metrics
- Business & developer impact
- DORA metrics awareness
- Continuous improvement
- Data-driven decisions

SUCCESS METRICS



⑦ Q: How would you design an IDP for a company?

- ANS:
- Understand developer needs & pain points.
 - Define MVP with core self-service capabilities.
 - Build golden paths and reusable components.
 - Iterate with feedback and measure impact.
 - Scale platform and evolve with the org.

INTERVIEWER LOOKS FOR:

- Structured approach
- Iterative & product mindset
- Collaboration
- MVP thinking
- Scalability & evolution

IDP DESIGN JOURNEY



KEY TAKEAWAY:

Platform Engineering is about building paved roads, not removing freedom.
Empower developers. Ship better software.

PLATFORM ENGINEERING MINDSET



SENIOR SYSTEM DESIGN QUESTIONS

Real interview questions. Real production scenarios.

TOPICS COVERED:

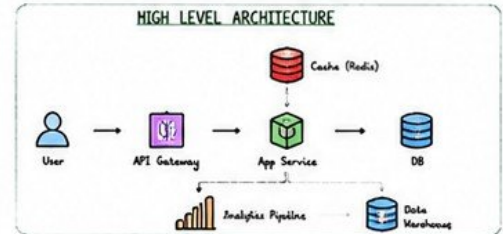
- ✓ Architecture decisions
- ✓ Trade-offs
- ✓ Large-scale systems

① Q: Design a URL shortener like bit.ly.

- ANS:
- Write path: User → API → Generate short code → Store mapping.
 - Read path: Short URL → API → Lookup → Redirect.
 - Use base62 encoding for short IDs.
 - Store in scalable DB (Cassandra / DynamoDB).
 - Cache hot mappings in Redis.
 - Track analytics (clicks, geo, device).

INTERVIEWER LOOKS FOR:

- Requirements clarification
- High level design
- Scalability & availability
- Data model
- Bottlenecks & optimizations

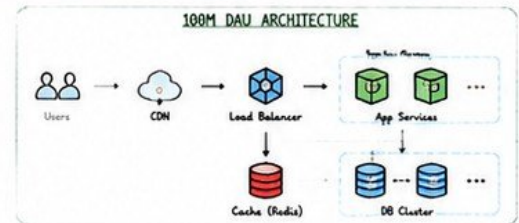


② Q: Design a system to handle 100M DAU (Daily Active Users).

- ANS:
- Use microservices with domain boundaries.
 - Stateless services behind load balancers.
 - Auto-scaling (horizontal) based on metrics.
 - Caching layer for reads (Redis / CDN).
 - Asynchronous processing for non-critical tasks.
 - Partition data by user_id / region.

INTERVIEWER LOOKS FOR:

- Scalability thinking
- Stateless & distributed design
- Data partitioning
- Caching strategies
- Resilience & fault tolerance

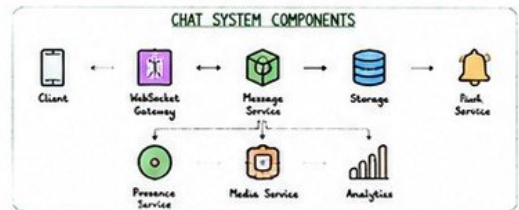


③ Q: How would you design a chat system like WhatsApp?

- ANS:
- Real-time messaging with WebSocket / MQTT.
 - Store messages in NoSQL + cold storage.
 - Presence service using in-memory store.
 - Delivery guarantees: ACKs & retries.
 - Scale by sharding on user_id.
 - End-to-end encryption for security.

INTERVIEWER LOOKS FOR:

- Real-time communication
- Data model & storage
- Reliability & ordering
- Scalability (sharding)
- Security considerations

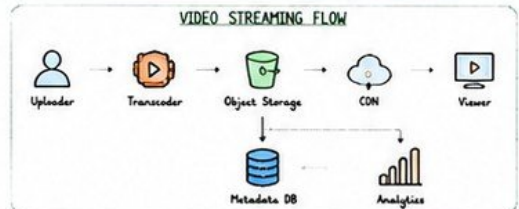


④ Q: Design a video streaming platform.

- ANS:
- Upload → Transcode → Store → CDN → View.
 - Use object storage (S3/GCS) for videos.
 - Use HLS/DASH for adaptive streaming.
 - CDN for global delivery.
 - Metadata in SQL / NoSQL DB.
 - Analytics for views, retention, quality.

INTERVIEWER LOOKS FOR:

- End-to-end flow
- Scalability & performance
- Storage & CDN usage
- Trade-offs (latency vs cost)
- Monitoring & analytics

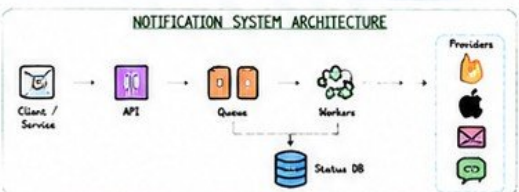


⑤ Q: How would you design a global notification system?

- ANS:
- Accept notifications → Queue → Workers → Delivery.
 - Use message queue (Kafka / SQS).
 - Channel providers: FCM, APNs, Email, SMS.
 - Retry with exponential backoff.
 - Track delivery status & failures.

INTERVIEWER LOOKS FOR:

- Decoupling with queues
- Reliability & retries
- Multi-channel support
- Idempotency
- Monitoring & alerting

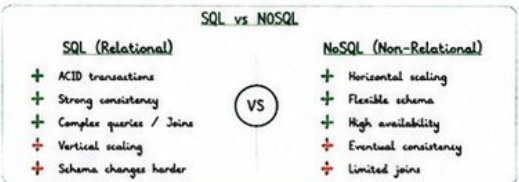


⑥ Q: What are the trade-offs between SQL and NoSQL?

- ANS:
- SQL: ACID, Joins, Strong consistency.
 - NoSQL: Scalability, Schema flexibility, High availability.
 - SQL is good for transactions.
 - NoSQL is good for scale and speed.
 - Choose based on data model & access pattern.

INTERVIEWER LOOKS FOR:

- Understanding trade-offs
- Use case alignment
- Consistency models
- Schema design thinking
- Practical judgement

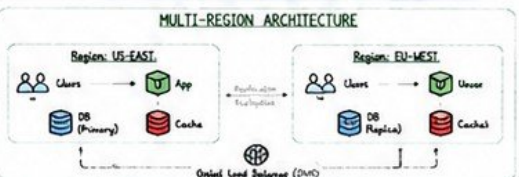


⑦ Q: How would you design for multi-region, low latency?

- ANS:
- Deploy active-active across regions.
 - Use global load balancing (Route53 / Anycast).
 - Data replication (async / active-active).
 - Cache closer to users.
 - Design for failure in a region.

INTERVIEWER LOOKS FOR:

- Global architecture
- Data replication strategies
- Latency optimization
- Failure handling
- User experience focus



KEY TAKEAWAY:

Great system design is about making the right trade-offs, not perfect design.
Clarify. Simplify. Scale. Iterate.

SENIOR DESIGN MINDSET



Clarify requirements



Explore options



Make trade-offs



Design for scale



Think failure



Measure & Improve

PRODUCTION MIGRATION QUESTIONS

Real interview questions. Real production scenarios.

TOPICS COVERED:

- ✓ Cloud migration
- ✓ Legacy systems
- ✓ Zero downtime migration

① Q: How would you migrate an on-prem application to the cloud?

- ANS:
- Assess & classify the application (dependencies, workload, data).
 - Choose migration strategy (rehost, replatform, refactor).
 - Build landing zone (network, security, IAM, logging).
 - Migrate in stages (dev → test → staging → prod).
 - Validate, optimize, cutover & monitor.

INTERVIEWER LOOKS FOR:

- Structured migration approach
- Risk & dependency assessment
- Cloud best practices
- Phased execution
- Post-migration optimization

CLOUD MIGRATION JOURNEY



② Q: What factors do you consider before migrating to the cloud?

- ANS:
- Business goals & expected benefits.
 - Application dependencies & compatibility.
 - Data size, sensitivity & compliance.
 - Performance, availability & scalability needs.
 - Cost & licensing implications.

INTERVIEWER LOOKS FOR:

- Business alignment
- Risk & compliance awareness
- Technical feasibility
- Cost thinking
- Long-term impact

MIGRATION READINESS CHECKLIST

- ✓ Application inventory
- ✓ Dependency mapping
- ✓ Data classification
- ✓ Security & compliance review
- ✓ Cost analysis (TCO)
- ✓ Skills & tooling readiness
- ✓ Rollback strategy

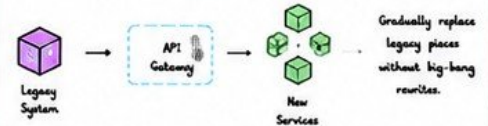
③ Q: How do you handle legacy systems during migration?

- ANS:
- Assess the system (tech stack, dependencies, risks).
 - Use strangler pattern to modernize incrementally.
 - Wrap/encapsulate with APIs or services.
 - Retain critical stable parts & replace gradually.
 - Monitor & manage risks throughout.

INTERVIEWER LOOKS FOR:

- Pragmatism
- Incremental modernization
- Risk management
- Working with constraints
- Business continuity focus

STRANGLER PATTERN



④ Q: How do you achieve zero downtime during migration?

- ANS:
- Use blue/green or canary deployments.
 - Database replication & dual-write strategy.
 - DNS / Load balancer traffic shifting.
 - Feature flags & dark launches.
 - Continuous monitoring & quick rollback plan.

INTERVIEWER LOOKS FOR:

- High availability mindset
- Traffic management
- Data consistency approach
- Validation & rollback
- Attention to detail

ZERO DOWNTIME MIGRATION FLOW



⑤ Q: What are the biggest risks in a migration and how do you mitigate them?

- ANS:
- Downtime → Use phased rollout & HA architecture.
 - Data loss → Backups, replication, data validation.
 - Performance issues → Load testing & tuning.
 - Security gaps → IAM, encryption, least privilege.
 - Cost overrun → Budgets, monitoring, right-sizing.

INTERVIEWER LOOKS FOR:

- Risk identification
- Mitigation strategies
- Balancing speed & safety
- Monitoring & validation
- Cost governance

TOP MIGRATION RISKS

- ⚠ Downtime
- ⚠ Data loss / Corruption
- ⚠ Performance degradation
- ⚠ Security vulnerabilities
- ⚠ Cost overrun

MITIGATION

- ✓ Phased rollout & HA
- ✓ Backups & validation
- ✓ Testing & optimization
- ✓ Security best practices
- ✓ Monitoring & budgets

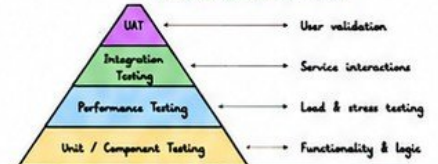
⑥ Q: How do you test a migration before going live?

- ANS:
- Test in lower environments (dev/test/stage).
 - Data integrity & consistency checks.
 - Performance & load testing.
 - Failover & disaster recovery testing.
 - User acceptance testing (UAT).

INTERVIEWER LOOKS FOR:

- Comprehensive testing
- Validation mindset
- Performance focus
- DR preparedness
- User-centered thinking

MIGRATION TESTING PYRAMID



⑦ Q: How do you roll back if something goes wrong?

- ANS:
- Keep old environment running until cutover complete.
 - Use feature flags to disable new features.
 - DNS / LB rollback to previous version.
 - Database point-in-time recovery.
 - Document rollback plan & test it.

INTERVIEWER LOOKS FOR:

- Rollback planning
- Safety nets
- Decisiveness under pressure
- Communication
- Tested procedures

ROLLBACK STRATEGY



KEY TAKEAWAY:

Migrations are not just technical moves.
They are **business transformations**.
Plan smart. Test hard. Migrate safe.

MIGRATION SUCCESS MINDSET



ADVANCED TROUBLESHOOTING SCENARIOS

Real interview questions. Real production scenarios.

TOPICS COVERED:

- ✓ Complex incidents
- ✓ Multiple failures
- ✓ Investigation workflow

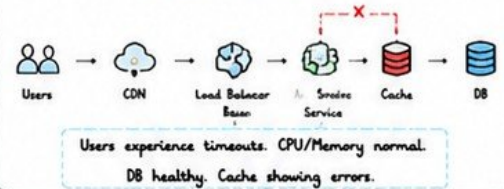
① Q: How do you approach a complex production incident?

- ANS:
- Stay calm and gather facts → not guesses.
 - Understand the impact and scope.
 - Identify what changed.
 - Form hypothesis → Test → Iterate.
 - Communicate clearly and update frequently.

INTERVIEWER LOOKS FOR:

- Structured thinking
- Prioritization
- Root cause mindset
- Communication
- Ownership & accountability

COMPLEX INCIDENT EXAMPLE



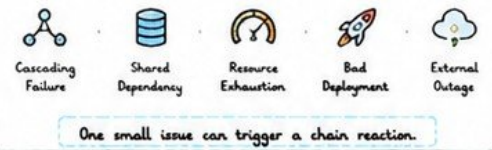
② Q: What are common causes of multiple simultaneous failures?

- ANS:
- Cascading failures (one service impacts others).
 - Shared dependency failures (DB, Cache, Network).
 - Resource exhaustion.
 - Bad deployments / config changes.
 - External provider outages.

INTERVIEWER LOOKS FOR:

- Systems thinking
- Understanding dependencies
- Root cause identification
- Experience with real-world issues
- Risk awareness

MULTIPLE FAILURE PATTERNS



③ Q: How do you identify the root cause in a complex incident?

- ANS:
- Collect data from all layers (App, Infra, Network, DB).
 - Build a timeline of events.
 - Look for patterns and correlations.
 - Eliminate possibilities.
 - Confirm root cause with evidence.

INTERVIEWER LOOKS FOR:

- Data-driven approach
- Correlation & pattern recognition
- Hypothesis validation
- Attention to detail
- Evidence-based conclusion

ROOT CAUSE TRIANGLE



④ Q: How do you communicate during a major incident?

- ANS:
- Acknowledge quickly.
 - Set expectations (updates, ETA).
 - Share facts, not assumptions.
 - Use simple language.
 - Close the loop with resolution & next steps.

INTERVIEWER LOOKS FOR:

- Clear communicator
- Stakeholder management
- Transparency
- Customer empathy
- Professionalism under pressure

COMMUNICATION FLOW



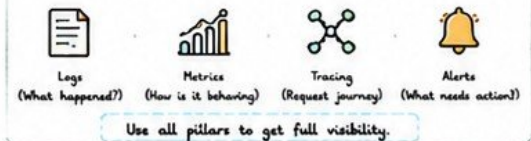
⑤ Q: What tools and data sources do you use during incidents?

- ANS:
- Logs: ELK, Loki, CloudWatch, Splunk
 - Metrics: Prometheus, Grafana, Datadog
 - Tracing: Jaeger, Zipkin, OpenTelemetry
 - Synthetic checks & monitors
 - Dashboards, Alerts, Runbooks

INTERVIEWER LOOKS FOR:

- Tool familiarity
- Observability mindset
- Data correlation skills
- Efficiency
- Practical experience

OBSERVABILITY PILLARS



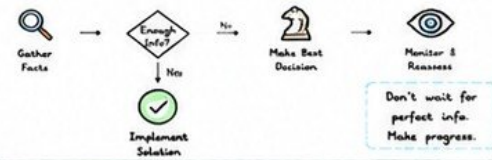
⑥ Q: How do you handle an incident with incomplete information?

- ANS:
- Work with what you have.
 - Make educated guesses but validate.
 - Prioritize actions that reduce risk.
 - Keep investigating in parallel.
 - Update stakeholders with what you know.

INTERVIEWER LOOKS FOR:

- Adaptability
- Risk management
- Critical thinking
- Decision making
- Calm under uncertainty

DECISION FRAMEWORK



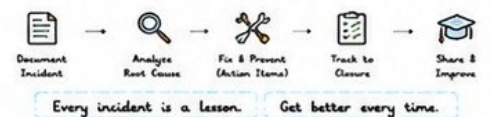
⑦ Q: What is your post-incident process?

- ANS:
- Write blameless postmortem.
 - Identify root causes & contributing factors.
 - Define corrective and preventive actions.
 - Track actions to closure.
 - Share learnings & improve systems.

INTERVIEWER LOOKS FOR:

- Continuous improvement
- Blameless culture
- Accountability
- Process orientation
- Learning mindset

POST-INCIDENT LIFECYCLE



KEY TAKEAWAY:

In complex incidents, tools help, but your mindset solves problems.
Stay Calm. Stay Curious. Stay Persistent.

TROUBLESHOOTING MINDSET



VERIQT

@veriqta

DEVOPS INTERVIEW CHEAT SHEET

Quick reference. Smart answers. Stand out.



HOW TO USE THIS SHEET

- Skim before interviews
- Use during whiteboard discussions
- Refer during real incidents
- Keep learning & stay sharp!

1 COMMON LINUX COMMANDS

PURPOSE

EXAMPLE

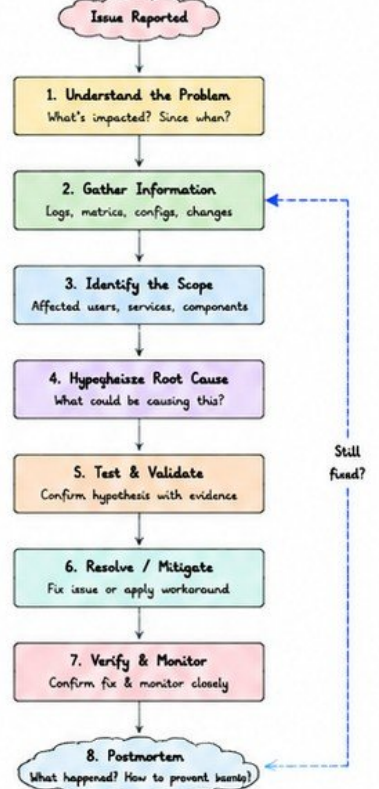
List files	ls -lah
Change directory	cd /path/to/dir
Print working dir	pwd
View file content	cat file.txt
Search in file	grep "keyword" file.txt
Find files	find / -name file.txt
Check disk usage	df -h
Check memory	free -m
Check processes	top / htop
Kill process	kill -9 PID
Show open ports	ss -tulnp
Test connectivity	ping google.com
View logs (last 100)	tail -n 100 /var/log/syslog
Extract archive	tar -xvf file.tar.gz
Change permissions	chmod 755 script.sh

2 DEVOPS TOOLS AT A GLANCE

EXAMPLE

Version Control	Git / GitHub / GitLab
CI/CD	Jenkins / GitLab CI / GitHub Actions
Containers	Docker / Podman
Container Orchestration	Kubernetes
Infrastructure as Code	Terraform / CloudFormation / Ansible
Configuration Management	Ansible / Puppet / Chef
Artifact Repository	JFrog Artifactory / Nexus
Code Quality	SonarQube
Monitoring	Prometheus + Grafana
Logging	ELK Stack (Elasticsearch, Logstash, Kibana)

3 TROUBLESHOOTING FLOW



4 GIT COMMANDS (MUST KNOW)

- git clone <repo> # Clone a repository
- git status # Check status
- git add . # Stage changes
- git commit -m "msg" # Commit changes
- git push origin <branch> # Push changes
- git pull # Pull latest changes
- git checkout -b <branch> # Create new branch
- git branch # List branches
- git merge <branch> # Merge branch
- git log --oneline --graph --all # null # View history

Tip: Always push before pull.
Keep commits small and meaningful.

5 DOCKER COMMANDS

- docker ps -a # List all containers
- docker images # List images
- docker pull <image> # Pull image
- docker run -d -p 80:80 <image> # Run container
- docker exec -it <cid> bash # Access container
- docker logs <cid> # View logs
- docker stop <cid> # Stop container
- docker rm <cid> # Remove container
- docker rmi <image> # Remove image
- docker build -t myimg . # Build image

Tip: Containers are immutable.
Rebuild > SSH and fix.

6 KUBERNETES COMMANDS

- kubectl get pods -A # All pods
- kubectl get svc -A # All services
- kubectl describe pod <pod> # Describe pod
- kubectl logs <pod> # Pod logs
- kubectl exec -it <pod> -- bash # Exec into pod
- kubectl get deployments # Deployments
- kubectl get nodes # Nodes
- kubectl top pods # Pod metrics
- kubectl apply -f file.yaml # Apply manifest
- kubectl delete -f file.yaml # Delete manifest

Tip: Kubernetes is declarative.
Define desired state, not steps.

INFRASTRUCTURE AS CODE (TERRAFORM)

- terraform init # Initialize
- terraform validate # Validate code
- terraform plan # Show execution plan
- terraform apply # Apply changes
- terraform destroy # Destroy infrastructure
- terraform fmt # Format code
- terraform graph # Generate graph
- terraform stack list # List resources
- terraform output # Show outputs
- terraform taint <res> # Mark resource for recreate

Tip: Plan before apply.
Use remote state. Never hardcode secrets.

7 ARCHITECTURE REMINDERS

THE 6 PILLARS OF RELIABLE SYSTEMS



ARCHITECTURAL PATTERNS

- Microservices
- Event-Driven
- CQRS
- Serverless
- API Gateway
- Circuit Breaker

DESIGN TRADE-OFFS



There is no perfect architecture.
Choose what fits the context.

8 INTERVIEW PRO TIPS

1. Clarify First
Ask questions. Understand requirements.
2. Think Out Loud
Explain your thought process step by step.
3. Start Simple
Begin with a basic solution, then improve.
4. Discuss Trade-offs
Show you understand pros, cons and alternatives.
5. Real World Mindset
Think about failure, monitoring, security and cost.

Stay calm. Be structured.
Show ownership.
You've got this!

REMEMBER

It's not about knowing everything.
It's about how you think, solve problems and learn.

SENIOR DEVOPS ENGINEER FINAL ASSESSMENT

You don't just deploy. You own production.

★ WHAT THIS IS

A quick assesment of the skills and mindset of a production-ready Senior DevOps Engineer.

Can you own it end-to-end?

① PRODUCTION READINESS

Can you ensure the system is reliable, resilient and observable?

- ☐ High availability & fault tolerance
- ☐ Scalability & performance planning
- ☐ Monitoring, alerting & logging
- ☐ Incident management & on-call readiness
- ☐ Backup strategy & disaster recovery
- ☐ Capacity planning & cost optimization
- ☐ Runbooks & operational documentation

READINESS SCORE

☆☆☆☆☆
☆☆☆☆☆
☆☆☆☆☆
☆☆☆☆☆
☆☆☆☆☆
☆☆☆☆☆
☆☆☆☆☆

Goal: Zero surprises in production.

② CLOUD SKILLS

Can you design, build and run on the cloud efficiently and securely?

- ☐ VPC, subnets, routing, security groups
- ☐ Compute (EC2, ASG, Lambda, ECS/EKS)
- ☐ Storage (S3, EBS, EFS, RDS, DynamoDB)
- ☐ Load balancing & Auto Scaling
- ☐ IAM, policies, roles & federation
- ☐ Multi-region & disaster recovery
- ☐ Cost management & rightsizing

CLOUD SCORE

☆☆☆☆☆
☆☆☆☆☆
☆☆☆☆☆
☆☆☆☆☆
☆☆☆☆☆
☆☆☆☆☆
☆☆☆☆☆

Goal: Secure, scalable and cost-effective cloud.

③ AUTOMATION SKILLS

Can you automate infrastructure, delivery and operations?

- ☐ Infrastructure as Code (Terraform/CloudFormation)
- ☐ Configuration Management (Ansible/Puppet/Chef)
- ☐ CI/CD Pipelines (Jenkins/GitLab/GitHub Actions)
- ☐ Automated testing & quality gates
- ☐ Artifact management (Nexus/JFrog)
- ☐ Release strategies (Blue/Green, Canary)
- ☐ ChatOps & task automation

AUTOMATION SCORE

☆☆☆☆☆
☆☆☆☆☆
☆☆☆☆☆
☆☆☆☆☆
☆☆☆☆☆
☆☆☆☆☆
☆☆☆☆☆

Goal: Automate everything that can be automated.

④ SECURITY SKILLS

Can you build secure systems and protect what matters?

- ☐ Identity & Access Management (IAM)
- ☐ Network security & segmentation
- ☐ Secrets management (Vault, KMS, SSM)
- ☐ Container & image security
- ☐ Vulnerability scanning & patching
- ☐ Compliance (CIS, SOC2, ISO, PCI, GDPR)
- ☐ Threat detection & security monitoring

SECURITY SCORE

☆☆☆☆☆
☆☆☆☆☆
☆☆☆☆☆
☆☆☆☆☆
☆☆☆☆☆
☆☆☆☆☆
☆☆☆☆☆

Goal: Security by design, not an afterthought.

⑤ ARCHITECTURE SKILLS

Can you design reliable, scalable and maintainable systems?

- ☐ Microservices vs Monolith decisions
- ☐ Event-driven architecture
- ☐ API design & best practices
- ☐ Caching strategies (Redis, CDN)
- ☐ Database design & scaling
- ☐ Integration patterns & messaging (SQS/Kafka)
- ☐ Trade-off analysis & architecture docs

ARCHITECTURE SCORE

☆☆☆☆☆
☆☆☆☆☆
☆☆☆☆☆
☆☆☆☆☆
☆☆☆☆☆
☆☆☆☆☆
☆☆☆☆☆

Goal: Build systems that scale and evolve.

OVERALL ASSESMENT



FINAL SCORE

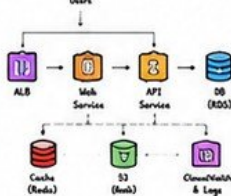
/ 50

RATING GUIDE

45 - 50	☆☆☆☆☆	Production Legend
35 - 44	☆☆☆☆☆	Strong Engineer
25 - 34	☆☆☆☆☆	Capable Engineer
15 - 24	☆☆☆☆☆	Keep Building
0 - 14	☆☆☆☆☆	Keep Learning

REAL-WORLD SCENARIO

Your company's e-commerce platform is experiencing slow performance and intermittent outages during peak traffic.



Your Mission:

- ✓ Investigate the issue
- ✓ Identify root cause
- ✓ Mitigate & resolve
- ✓ Prevent it from happening again

WHAT YOU WILL BE EVALUATED ON

- ✓ Problem solving & root cause analysis
- ✓ System design & decision making
- ✓ Command of tools & technologies
- ✓ Security & compliance awareness
- ✓ Communication & documentation
- ✓ Automation & efficiency mindset
- ✓ Ownership & reliability

It's not just about technical skills.
It's about how you think, decide and own results.

KEY QUESTIONS YOU SHOULD BE READY FOR

- ? How would you handle a production outage at 2 AM?
- ? How do you design for high availability & disaster recovery?
- ? How do you ensure secure access to cloud resources?
- ? How do you reduce cloud costs without impacting performance?
- ? How do you automate a full CI/CD pipeline?
- ? How do you handle breaking changes in production?
- ? How do you measure the success of your systems?



COMMAND SNAPSHOT (EXAMPLES)

Linux	top, df -h, free -m, netstat -tuln
Docker	docker ps -a, docker logs <id>, docker exec -it <id> bash
Kubernetes	kubectl get pods -A, kubectl describe pod <pod>, kubectl logs -f <pod>
AWS CLI	aws s3 ls, aws ec2 describe-instances, aws sts get-caller-identity
Terraform	terraform plan, terraform apply, terraform destroy
Git	git status, git log --oneline, git branch -a, git merge <branch>

SUCCESS FORMULA

Knowledge + Tools + Automation + Security + Architecture + Ownership
 = RELIABLE PRODUCTION SYSTEMS

MINDSET CHECK

- ☐ I think like an owner, not just a deployer.
- ☐ I simplify complexity.
- ☐ I automate & document.
- ☐ I learn from every incident.
- ☐ I build for the future.

YOU GOT THIS!



FINAL REMINDER

Systems break. That's normal.
It's how you design, respond and improve that makes you a Senior Engineer.



KEEP BUILDING. KEEP LEARNING. KEEP OWNING.
YOU ARE THE RELIABILITY.

VERIQTA

@veriqta